

SC1005 Digital Logic
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	Number Systems: From Counting to Binary and Hexadecimal	1
1.1	Why Not Decimal Inside a Computer?	1
1.2	Conversions	2
1.3	Unsigned Range and Overflow	3
1.4	Signed Numbers and Two’s Complement	3
1.5	Binary Codes and Arithmetic Preview	4
1.6	Fractions, Floating-Point Preview, and Binary Arithmetic	4
1.7	Chapter Notes	4
1.8	Fixed-Point vs. Floating Preview	4
1.9	Exercises	4
2	Boolean Algebra and Karnaugh Maps	6
2.1	Boolean Algebra: Axioms and Intuition	6
2.2	Canonical Forms: SOP and POS	7
2.3	Karnaugh Maps	7
2.4	Chapter Notes	8
2.5	Exercises	8
3	Combinational Logic: Adders, Multiplexers, Decoders	9
3.1	Combinational Building Blocks	9
3.2	Adders: Half, Full, Ripple-Carry	10
3.3	Multiplexers and Demultiplexers	10
3.4	Decoders, Encoders, and Tri-State	10
3.5	Comparators, Code Converters, and Hierarchical Design	11

3.6	Chapter Notes	11
3.7	Timing and Hazards in Combinational Logic	11
3.8	Exercises	12
4	Sequential Logic: Latches and Flip-Flops	13
4.1	Why Feedback? From Combinational to Sequential	13
4.2	SR Latch: The Simplest Memory	14
4.3	D Latch: Removing the Forbidden State	14
4.4	From Latch to Flip-Flop	14
4.5	Timing Analysis and Flip-Flop Variants in Practice	15
4.6	Chapter Notes	15
4.7	Metastability and Synchronizers	15
4.8	Exercises	16
5	Finite State Machines: Mealy, Moore, and Design	17
5.1	What Is a Finite State Machine?	17
5.2	State Diagrams and Tables	18
5.3	Synchronous Synthesis	18
5.4	State Encoding and Mealy–Moore Interconversion	19
5.5	Chapter Notes	19
5.6	ASM Charts and Algorithmic State Machines	19
5.7	Exercises	20
6	Registers and Counters	21
6.1	Registers: Parallel Storage	21
6.2	Shift Registers	22
6.3	Counters	22
6.4	Counter Modulus, Self-Correction, and Timing	23
6.5	Chapter Notes	23
6.6	Applications: From Counters to Timers	23
6.7	Exercises	24

7	Arithmetic Circuits: Fast Adders, Multipliers, and ALUs	25
7.1	Why Ripple Is Too Slow	25
7.2	Carry-Lookahead Adder (CLA)	26
7.3	Subtraction and Overflow in Hardware	26
7.4	Multiplication	26
7.5	Arithmetic Logic Unit (ALU)	27
7.6	Carry-Select, Prefix Adders, and Multiplication Tradeoffs	27
7.7	Chapter Notes	27
7.8	Division Preview and Fixed-Point Scaling	28
7.9	Exercises	28
8	Memory and Programmable Logic	30
8.1	Memory Organization	30
8.2	SRAM vs. DRAM Cell	31
8.3	Programmable Logic: PLA, PAL, ROM	31
8.4	FPGAs: Logic You Configure	31
8.5	Read/Write Timing and Memory Expansion	32
8.6	Chapter Notes	32
8.7	Memory Hierarchy and Locality	32
8.8	Exercises	33

Chapter 1

Number Systems: From Counting to Binary and Hexadecimal

“There are 10 kinds of people: those who understand binary and those who don’t.” Digital hardware speaks only two voltages. This chapter teaches you to count, compute, and think fluently in that language, starting from zero—no prior digital background assumed.

From zero: built for absolute beginners. We assume only school arithmetic. Every idea is picture $\rightarrow$ words $\rightarrow$ definition. By the end you will convert, add, and negate in binary/hex as naturally as in decimal.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain positional number systems and weight of each digit position;
- (L2) convert fluently among decimal, binary, octal, and hexadecimal;
- (L3) count binary ranges and detect overflow for n -bit unsigned numbers;
- (L4) represent signed integers via sign-magnitude, one’s and two’s complement;
- (L5) add, subtract, and reason about overflow in two’s complement arithmetic.

1.1 Why Not Decimal Inside a Computer?

Decimal uses ten symbols; hardware would need ten distinct voltage levels—fragile against noise. Two levels (LOW ≈ 0 V, HIGH $\approx V_{DD}$) are maximally noise-immune. One binary digit (bit) stores one yes/no choice. Eight bits make a byte, the standard storage atom. This is why all modern computers are binary at the bottom.

Positional intuition. In decimal $472 = 4 \times 10^2 + 7 \times 10^1 + 2 \times 10^0$. Replace base 10 by 2: each position weighs twice the one to its right. Same rules, smaller base. The rightmost position is the least-significant bit (LSB), the leftmost the most-significant bit (MSB).

Definition 1.1 (Positional representation). In base (radix) r , digits $d_i \in \{0, \dots, r-1\}$ and

$$N = d_{k-1}r^{k-1} + \dots + d_1r^1 + d_0r^0 + \dots + d_{-1}r^{-1} + \dots$$

We write $(d_{k-1} \dots d_0.d_{-1} \dots)_r$. Decimal ($r = 10$), binary ($r = 2$), octal ($r = 8$), hexadecimal ($r = 16$, digits 0–9, A–F) are the four we use.

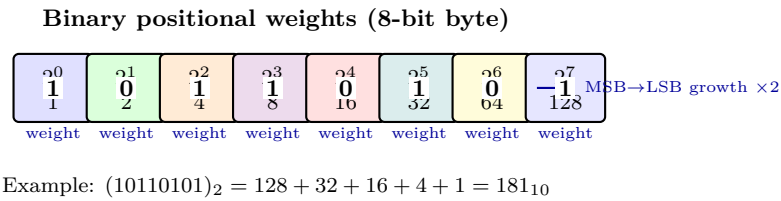


Figure 1.1: Each binary position weighs a power of two. The rightmost bit (LSB) weighs 1, next 2, then 4, 8, ...

1.2 Conversions

Binary $\rightarrow$ decimal: sum weights of 1-bits. Example: $(1101)_2 = 8 + 4 + 1 = 13$.

Decimal $\rightarrow$ binary: repeated division by 2, read remainders bottom-up. $13 \div 2 = 6$ r1, $6 \div 2 = 3$ r0, $3 \div 2 = 1$ r1, $1 \div 2 = 0$ r1 $\Rightarrow 1101$. For fractions, repeated multiply by 2 and collect integer parts.

Hexadecimal shortcut. Since $16 = 2^4$, group bits in fours: $(1011\ 0110)_2 = (B6)_{16}$, and $(3F)_{16} = (0011\ 1111)_2$. This is why hex is ubiquitous in datasheets, addresses, and machine code listings—compact alias for binary.

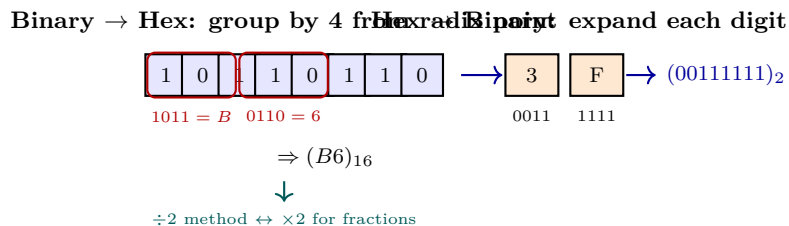


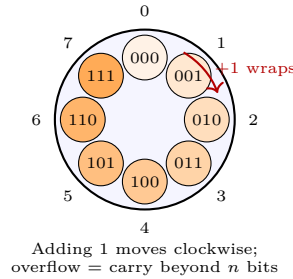
Figure 1.2: Binary–hex conversion is lossless grouping by four bits; decimal conversion uses repeated division/multiplication.

Example 1.1 (Conversions). Convert $(2A.4)_{16}$ to decimal: $2 \times 16 + 10 + 4/16 = 42.25$. Convert 42.25 to binary: $42 = (101010)_2$, $0.25 = (.01)_2$, so 101010.01_2 —check hex 2A.4 matches. Octal groups by 3 bits: $(101101)_2 = (55)_8$ since $16 = 2^4$, $8 = 2^3$.

1.3 Unsigned Range and Overflow

An n -bit unsigned integer spans 0 to $2^n - 1$. With $n = 8$, 0–255; $n = 32$, 0–4 294 967 295. Adding 1 to $111 \dots 1$ wraps to $00 \dots 0$ (overflow / carry out). Hardware signals this with a carry flag.

3-bit unsigned wheel: 0–7 then wrap



Unsigned range

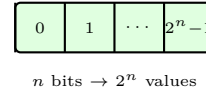


Figure 1.3: Unsigned n -bit numbers live on a wrap-around wheel. The extra carry beyond n bits is the overflow flag.

1.4 Signed Numbers and Two's Complement

Three historical schemes for signed n -bit integers:

- **Sign-magnitude:** MSB is sign (0 = +, 1 = −), rest is magnitude. Two zeros! Range $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$.
- **One's complement:** Negate by flipping all bits. Also two zeros. Rare today.
- **Two's complement** (universal): Negate by invert+1. One zero, range -2^{n-1} to $2^{n-1} - 1$. Addition is same as unsigned—hardware's reason for winning.

Definition 1.2 (Two's complement). For n bits, value of bit-vector $b_{n-1} \dots b_0$ is

$$V = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

Negation: $-N = \overline{N} + 1$ (bitwise NOT plus one). Subtracting $A - B$ is $A + \overline{B} + 1$.

Example 1.2 (Two's complement arithmetic). $n = 4$: $+3 = 0011$, $-3 = 1101$ (invert $0011 \rightarrow 1100+1$). $3 + (-3) = 0011 + 1101 = 0000$ (carry discarded) correct. $5 + 6 = 0101 + 0110 = 1011$ which is -5 after wrap—signed overflow since two positives yielded negative. 4-bit range is -8 to $+7$; 1011 as signed is -5 , as unsigned is 11 .

Theorem 1.1 (Overflow rule). Signed overflow in two's complement addition occurs iff carry into sign bit $\neq$ carry out of sign bit, equivalently: adding two same-sign numbers yields opposite sign. Unsigned overflow is simply carry out = 1.

1.5 Binary Codes and Arithmetic Preview

Not all bit patterns are numbers. **BCD** encodes each decimal digit in 4 bits (wasteful but decimal-exact, used in calculators). **Gray code** changes one bit per step—ideal for rotary encoders and Karnaugh maps (no glitch). **ASCII/Unicode** map bit patterns to characters; $(41)_{16} = \text{'A'}$, $(30)_{16} = \text{'0'}$. Binary addition is column-wise with carries, exactly like decimal but base 2: $1 + 1 = 0$ carry 1, $1 + 1 + 1 = 1$ carry 1.

1.6 Fractions, Floating-Point Preview, and Binary Arithmetic

Binary fractions mirror decimal: $(0.101)_2 = 1/2 + 1/8 = 0.625$. Not every decimal fraction terminates in binary ($0.1_{10} = 0.00011\overline{0011}_2$ recurring), which is why floating-point rounding matters—preview for computer arithmetic courses. Binary addition is column-wise:

$$\begin{array}{rcccc}
 & 1 & 0 & 1 & 1 \\
 + & 0 & 1 & 1 & 0 \\
 \hline
 \text{carry} & 1 & 1 & 0 & 0 \\
 \text{result} & 1 & 0 & 0 & 0 & 1
 \end{array}$$

Carry propagates left; the extra carry beyond n bits is exactly the unsigned overflow flag C . Subtraction via two's complement: $A - B = A + \bar{B} + 1$, so one adder does both operations with an invert-and-carry-in control.

Example 1.3 (Overflow table). $n = 4$ signed: $7 + 1 = 0111 + 0001 = 1000$ (-8) overflow; $-8 - 1 = 1000 + 1111 = 0111$ ($+7$) overflow. Unsigned, $15 + 1 = 1111 + 0001 = 0000$ carry 1—correct modulo 2^4 wrap.

1.7 Chapter Notes

Positional notation dates to Hindu-Arabic mathematics; binary championed by Leibniz (1703). Two's complement emerged with early stored-program machines (EDSAC, 1949) for its single-adder economy. Hex became standard with the IBM System/360 (1964) to print 32-bit words compactly.

1.8 Fixed-Point vs. Floating Preview

Fixed-point binary $(b_{k-1} \dots b_0.b_{-1} \dots b_{-m})_2$ has uniform step 2^{-m} and range $\approx 2^k$. More bits after the point = finer resolution; more before = larger range. For large dynamic range, floating-point (sign $\times$ mantissa $\times 2^{\text{exponent}}$) trades uniform precision for wide range—full treatment in computer arithmetic courses. Key takeaway: not all reals are representable; rounding error is inevitable and must be bounded (e.g. unit in last place, ULP).

1.9 Exercises

E1.1 **Conversions.** Convert $(101101.101)_2$ to decimal, hex, and octal. Convert $(3A7)_{16}$ and $(275)_8$ to binary. Show steps (division/multiplication).

- E1.2 **Two's complement.** For $n = 8$, give binary for $+73$, -73 , -128 , $+127$. Compute $73 + (-98)$ in binary and flag signed and unsigned overflow.
- E1.3 **Range and wrap.** An $n = 5$ unsigned counter increments by 1 from 0. After how many increments does it first read 0 again? Repeat for signed two's complement starting at $+15$ and incrementing.
- E1.4 **Hex dump reading.** A memory dump shows DE AD BE EF. Write as binary, as four unsigned bytes decimal, and as two 16-bit big-endian unsigned values. What about little-endian?
- E1.5 **Gray vs. binary.** Construct 3-bit Gray code, show adjacent words differ by one bit, and explain why a 2-bit transition $011 \rightarrow 100$ in binary could briefly read 111 on skewed wires.

Chapter 2

Boolean Algebra and Karnaugh Maps

“George Boole made logic into algebra; Shannon made algebra into circuits.” This chapter turns truth tables into equations, equations into minimal circuits, and minimal circuits into patterns you can see on a map—no prior logic assumed.

From zero: logic from scratch. We define propositions, connectives, and axioms before any algebra. Each law gets a Venn or gate picture first, then the identity.

Learning Outcomes

After this chapter you will be able to:

- (L1) evaluate and manipulate Boolean expressions using axioms and theorems;
- (L2) obtain canonical SOP/POS forms from truth tables;
- (L3) simplify functions algebraically and with Karnaugh maps up to 4 variables;
- (L4) identify prime implicants, essential primes, and don't-care handling;
- (L5) describe NAND/NOR universality and two-level cost metrics.

2.1 Boolean Algebra: Axioms and Intuition

A *Boolean variable* takes values in $\{0, 1\}$ (also $\{\text{false}, \text{true}\}$). Operations: $\cdot$ (AND), $+$ (OR), $'$ or $-$ or $\neg$ (NOT). Think sets: AND=intersection, OR=union, NOT=complement—the Venn picture *is* the law.

Definition 2.1 (Huntington axioms). For all $x, y, z \in \{0, 1\}$ with constants 0, 1: $x + 0 = x$, $x \cdot 1 = x$ (identity); $x + x' = 1$, $x \cdot x' = 0$ (complement); $x + y = y + x$, $x \cdot y = y \cdot x$ (commutative); $x + (y \cdot z) = (x + y) \cdot (x + z)$ and $x \cdot (y + z) = x \cdot y + x \cdot z$ (distributive); plus closure. Duality: swap $+$ $\leftrightarrow$ $\cdot$ and $0 \leftrightarrow 1$ to get a dual theorem for free.

Theorem 2.1 (Basic theorems). Idempotent $x + x = x$, $x \cdot x = x$; null $x + 1 = 1$, $x \cdot 0 = 0$; involution $(x')' = x$; absorption $x + x \cdot y = x$, $x \cdot (x + y) = x$; De Morgan $(x + y)' = x' \cdot y'$, $(x \cdot y)' = x' + y'$; consensus $x \cdot y + x' \cdot z + y \cdot z = x \cdot y + x' \cdot z$.

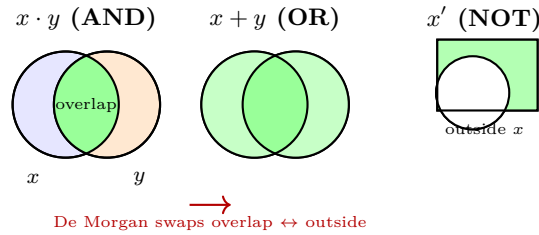


Figure 2.1: Venn intuition: AND is overlap, OR is union, NOT is outside. De Morgan's laws are visually obvious.

2.2 Canonical Forms: SOP and POS

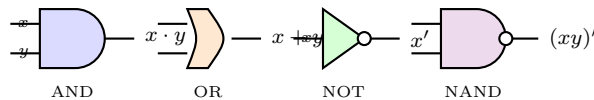
A *minterm* m_i is an AND of all literals (each variable complemented or not) that is 1 on exactly one row. A *maxterm* M_i is an OR that is 0 on exactly one row. Example for x, y, z : $m_5 = \bar{x}y\bar{z}$? no—with order $x y z$, $m_5 = x' \cdot y \cdot z'$? Check: binary 101 = 5.

Any function has two canonical forms:

$$F = \sum m(\text{rows where } F = 1) \quad (\text{SOP}), \quad F = \prod M(\text{rows where } F = 0) \quad (\text{POS}).$$

We use $\sum m(1, 3, 6)$ as shorthand.

Basic gates (color = function family)



Bubbled output = complemented; NAND/NOR are universal (any function from one gate type)

Figure 2.2: Gate symbols with color families. The small bubble denotes inversion; NAND follows AND with a bubble.

2.3 Karnaugh Maps

Algebraic minimization is error-prone. A *Karnaugh map* places minterms so adjacent cells differ by one literal (Gray order). Grouping 2^k adjacent 1s eliminates k literals.

Rules: groups must be rectangular, size power of two, wrap around edges, cover all 1s, make as large and as few as possible. A *prime implicant* cannot be enlarged; an *essential* one covers a 1 no other prime covers.

Don't-cares. Input combinations that never occur are marked $\times$ and may be taken as 0 or 1 whichever yields larger groups—free simplifiers.

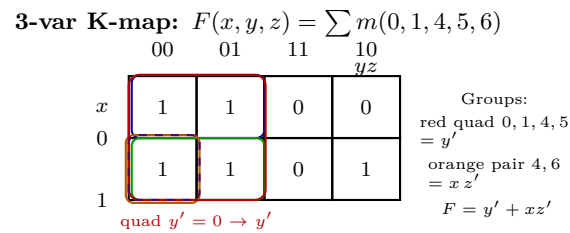


Figure 2.3: K-map with Gray-ordered columns. The red quad of four eliminates two literals to y' ; the pair 4–6 gives xz' . Result $F = y' + xz'$. Adjacent includes wrap-around.

Two-level cost. After minimization, cost $\approx$ number of gates + literals. Fewer product terms and fewer literals per term wins for SOP (and dually for POS).

2.4 Chapter Notes

Boole's *Laws of Thought* (1854) and Shannon's 1937 master's thesis linking switching circuits to Boolean algebra. Karnaugh (1953) and Veitch diagrams systematized minimization before Quine–McCluskey automation.

2.5 Exercises

E2.1 **Algebra.** Simplify $F = xy + x'z + yz$ to $xy + x'z$ using consensus, then verify by truth table.

E2.2 **Canonical forms.** For $F(a, b, c) = \sum m(0, 2, 3, 5, 7)$, write canonical POS and simplify via K-map to minimal SOP. List prime implicants and essential primes.

E2.3 **Don't-cares.** $F(w, x, y, z) = \sum m(1, 3, 7, 11, 15) + \sum d(0, 2, 5)$. Minimize with don't-cares; show how treating each $\times$ as 1 or 0 changes the cover.

E2.4 **NAND universality.** Using only 2-input NAND gates, realize $F = ab + cd$. Draw the gate diagram and count gates.

E2.5 **Duality check.** State the dual of absorption $x + xy = x$ and prove it. Use duality to obtain the POS form from an SOP minimization without remapping.

Chapter 3

Combinational Logic: Adders, Multiplexers, Decoders

“Combinational means memoryless—output is a pure function of present inputs.” From gates we build arithmetic, selection, and routing—the data-path building blocks every processor uses. No sequential knowledge assumed; we start from truth tables.

From zero: gates → blocks. We define combinational vs. sequential, then derive each block from its truth table → Boolean expression → circuit.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish combinational from sequential circuits;
- (L2) design half/full adders and ripple-carry adders from truth tables;
- (L3) use multiplexers as logic and routing elements;
- (L4) decode addresses with decoders and encode with priority encoders;
- (L5) analyse hazards and two-level vs. multi-level cost.

3.1 Combinational Building Blocks

A *combinational* circuit has no memory: output at time t depends only on inputs at t . No feedback, no clock. Given n inputs and m outputs it realises a Boolean function $f : \{0,1\}^n \rightarrow \{0,1\}^m$, fully specified by a truth table.

Design flow: (1) truth table, (2) SOP/POS or K-map to minimise (Ch. 2), (3) map to available gates (NAND/NOR universality), (4) check delay and hazards.

3.2 Adders: Half, Full, Ripple-Carry

Adding is the first non-trivial combinational function. One column of binary addition produces a sum bit and a carry.

Definition 3.1 (Half and full adder). *Half adder* (HA): inputs a, b ; outputs $S = a \oplus b$, $C_{\text{out}} = a \cdot b$ (adds two bits, no carry-in). *Full adder* (FA): inputs a, b, C_{in} ; $S = a \oplus b \oplus C_{\text{in}}$, $C_{\text{out}} = ab + C_{\text{in}}(a \oplus b)$.

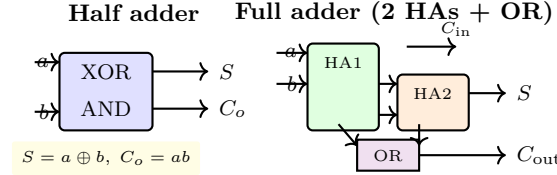


Figure 3.1: Left: half adder. Right: full adder built from two half adders plus OR for the carry. Coloured blocks match function.

Chaining FAs gives a *ripple-carry adder* (RCA): C_{out} of bit i feeds C_{in} of bit $i + 1$. Simple but carry ripples—delay $O(n)$ gate levels. Faster adders (carry-lookahead, Ch. 7) compute carries in parallel.

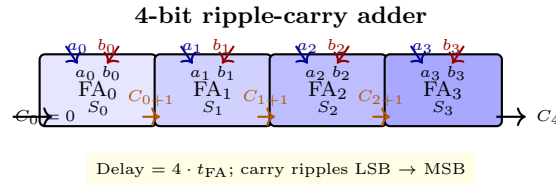


Figure 3.2: Ripple-carry adder: four full adders chained. The carry path (orange) determines critical delay.

3.3 Multiplexers and Demultiplexers

A $2^n \times 1$ *multiplexer* (MUX) selects one of 2^n data inputs via n select lines; a *demultiplexer* does the reverse. A MUX is a universal logic block: any n -variable function can be realised by wiring its truth table to data inputs and variables to selects.

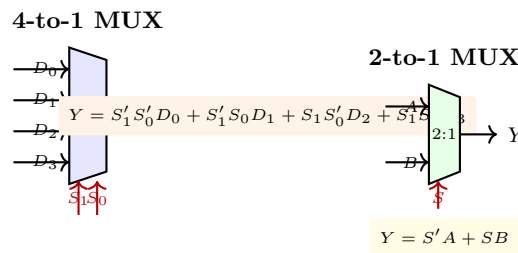


Figure 3.3: Multiplexers: 4:1 selects one of four data lines; 2:1 realises $Y = S'A + SB$. Select lines in red.

3.4 Decoders, Encoders, and Tri-State

A n -to- 2^n *decoder* asserts exactly one of 2^n outputs for each input code (address decoding). A *priority encoder* reverses this, reporting the highest-priority active input. *Tri-state* buffers allow bus sharing: output is 0, 1, or high-impedance Z (disconnected).

Hazards: combinational glitches when inputs change (static hazard: momentary 0 or 1 spike). K-map grouping with overlapping prime implicants or adding hazard covers fixes it for two-level logic.

3.5 Comparators, Code Converters, and Hierarchical Design

Beyond adders and MUXes, two more combinational staples appear everywhere. A *magnitude comparator* reports $A > B$, $A = B$, $A < B$ via subtraction without storing the difference: $A - B$ sign and zero flags give the answer. A *code converter* (e.g. binary to Gray or BCD to 7-segment) is simply a truth table mapped to SOP with don't-cares for unused codes.

Hierarchical design builds large blocks from small: an 8-bit RCA from two 4-bit RCAs, a 16-to-1 MUX from five 4-to-1 MUXes, a 5-to-32 decoder from 2-to-4 plus 3-to-8 decoders with enables. Enables ($\overline{EN}$) are the glue—inactive blocks output 0 or Z , so only the selected block drives the bus.

Example 3.1 (7-segment decoder). Hex digit $h_3h_2h_1h_0$ drives segments a – g . Row $h = 0000$ lights a, b, c, d, e, f but not g (displays '0'). K-maps for each segment share product terms—multi-output minimization saves gates versus per-output solo maps.

3.6 Chapter Notes

Claude Shannon's switching algebra (1938) first mapped Boolean functions to relay circuits. MUX-based design became popular with MSI chips (74151, 74138) in the 1970s TTL era.

3.7 Timing and Hazards in Combinational Logic

Gate delay matters. The *critical path* is the longest input-to-output chain; its delay sets the minimum clock period for the surrounding sequential system. A *static hazard* is a momentary glitch when one input changes but the output should stay constant—caused by unequal path delays reconverging (e.g. $F = x\bar{x}$ momentarily 1 during transition). A *dynamic hazard* toggles multiple times before settling.

K-maps naturally reveal static hazards: any adjacent 1s not covered by a common implicant is a potential glitch when crossing between them. Adding the *consensus* implicant that bridges the adjacency removes the hazard (at the cost of one extra term). Modern synthesis with multi-level logic may still introduce hazards—hazard-free design or registered outputs (Ch. 4) solves it.

Bus and tri-state. Multiple combinational blocks sharing a bus use *tri-state* buffers (0/1/ Z). Only one driver is enabled at a time; the bus is a wired-OR of tri-states. Modern CMOS prefers MUX-based buses internally (lower contention risk) with tri-states reserved for off-chip or bidirectional pins. Forgetting the enable decode is a classic lab bug—two drivers fighting draws large shoot-through current.

3.8 Exercises

- E3.1 **Full adder proof.** From the truth table derive S and C_{out} expressions, then show $C_{\text{out}} = ab + C_{\text{in}}(a \oplus b) = ab + aC_{\text{in}} + bC_{\text{in}}$.
- E3.2 **Ripple delay.** A FA has delay 2 gate levels for S and 1 for C_{out} . What is worst-case delay of an 8-bit RCA? Where is the critical path?
- E3.3 **MUX as logic.** Realise $F(x, y, z) = \sum m(1, 2, 5, 6)$ using (a) an 8:1 MUX, (b) a 4:1 MUX with x as data, (c) only 2:1 MUXes.
- E3.4 **Decoder expansion.** Build a 4-to-16 decoder from 2-to-4 decoders and gates. How many 2-to-4 blocks are needed? Draw the enable wiring.
- E3.5 **Hazard.** $F = xy + x'z$ glitches when $y = z = 1$ and x toggles. Show the K-map, identify the hazard, and add the consensus term yz to remove it.

Chapter 4

Sequential Logic: Latches and Flip-Flops

“Combinational circuits forget; sequential circuits remember.” Add feedback and you get memory. This chapter builds memory from first principles: SR latch $\rightarrow$ D latch $\rightarrow$ master-slave $\rightarrow$ edge-triggered flip-flop.

From zero: memory from wires. We assume only gates (Ch. 2–3). We add one idea—feedback—and derive every latch/flop from it. No prior sequential knowledge.

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish combinational from sequential and explain the role of feedback;
- (L2) analyse SR and D latches built from NAND/NOR gates;
- (L3) explain master-slave and edge-triggered D and JK flip-flops;
- (L4) describe setup/hold, propagation delay, and the clock concept;
- (L5) use characteristic tables and excitation tables for analysis.

4.1 Why Feedback? From Combinational to Sequential

A combinational output depends only on *present* inputs. Add a wire from output back to input and output now depends on *history*—that is memory. The simplest feedback loop is two cross-coupled inverters (a bistable): two stable states 0 and 1, holding one bit forever until forced to flip. All latches/flops elaborate this bistable core with control inputs.

A *clock* (periodic square wave) disciplines when state may change, avoiding races. *Level-sensitive* (latch) follows the input while the clock is high; *edge-triggered* (flip-flop) samples only on a clock edge ($\uparrow$ rising, $\downarrow$ falling).

4.2 SR Latch: The Simplest Memory

Cross-coupled NOR (or NAND) gates give Set-Reset behaviour: $S = 1$ sets $Q = 1$, $R = 1$ resets $Q = 0$, $S = R = 0$ holds, $S = R = 1$ is forbidden (both outputs try to be 0, metastable when released).

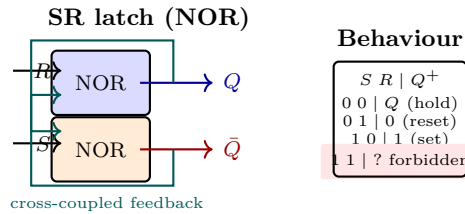


Figure 4.1: SR latch from cross-coupled NOR gates. Teal wires are feedback—the memory. Forbidden $S = R = 1$ breaks bistability.

Replacing NOR with NAND plus active-low inputs gives $\bar{S}\bar{R}$ latch, the building block of many CMOS latches. Adding an enable C gives the *gated SR latch*: when $C = 0$ it holds; when $C = 1$ it is transparent.

4.3 D Latch: Removing the Forbidden State

The D (data) latch ties $R = \bar{D}$ so S and R never both 1. One data input, one enable: $C = 1$ transparent ($Q = D$), $C = 0$ hold. Solves SR's forbidden state at the cost of one lost input combination.

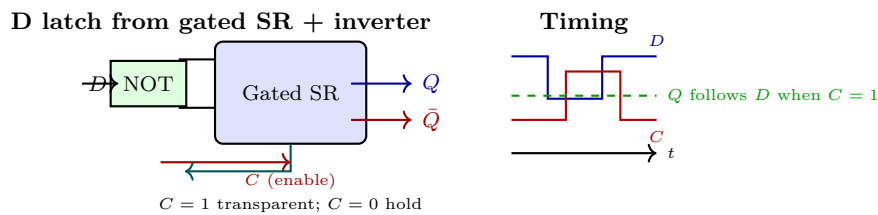


Figure 4.2: D latch: one inverter eliminates the forbidden state. When $C = 1$, Q tracks D ; when $C = 0$, Q holds.

Transparency problem. A latch is *level-sensitive*: while $C = 1$, any glitch on D propagates to Q . In multi-bit systems this causes races. The fix: make storage edge-triggered.

4.4 From Latch to Flip-Flop

Master-slave pairs two D latches in series on opposite clock phases: master transparent when $CLK = 0$, slave when $CLK = 1$ —data ripples through only on the $0 \rightarrow 1$ edge. Modern *edge-triggered D flip-flop* (DFF) achieves the same with fewer transistors (6 NAND gates) and is the standard library cell.

JK and T flip-flops. JK generalises SR: J sets, K resets, $J = K = 1$ toggles ($Q^+ = \bar{Q}$). No forbidden state. T (toggle) has one input: $T = 1$ toggles, $T = 0$ holds—ideal for counters (Ch. 6).

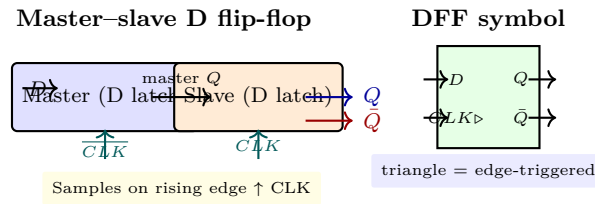


Figure 4.3: Master-slave construction and the standard D flip-flop symbol (triangle denotes edge-triggered clock).

Timing constraints. *Setup time* t_{su} : D must be stable before the edge. *Hold time* t_h : D must remain stable after the edge. *Clock-to- Q* t_{cq} : delay from edge to new Q . Violating t_{su}/t_h risks *metastability* (output hovers mid-rail).

Definition 4.1 (Characteristic and excitation tables). *Characteristic*: Q^+ as function of inputs and Q . D: $Q^+ = D$; JK: $Q^+ = JQ' + \bar{K}Q$. *Excitation*: inputs required to get $Q \rightarrow Q^+$. D needs $D = Q^+$; JK needs J, K pattern per transition.

4.5 Timing Analysis and Flip-Flop Variants in Practice

Real flip-flops have *propagation delay* t_{cq} (clock to Q), *setup* t_{su} and *hold* t_h . For a register-to-register path with combinational delay t_{comb} ,

$$T_{clk} \geq t_{cq} + t_{comb} + t_{su}, \quad t_{hold} \leq t_{cq} + t_{comb}^{\min}.$$

If T_{clk} is too short, setup fails; if $t_{comb}^{\min}$ is too short, hold fails (often fixed by buffer insertion). *Asynchronous* pre-set/clear override the clock—useful for power-on reset but must be deasserted synchronously to avoid metastability.

Example 4.1 (Choosing a flip-flop). D is simplest for data-path registers (one data pin, no decode). JK/T shine where toggling is the natural operation (counters, frequency dividers: $f_{out} = f_{clk}/2^n$ from n cascaded T stages). In CMOS libraries, DFF is the default cell; JK is built as D with feedback when needed.

4.6 Chapter Notes

The bistable latch dates to Eccles–Jordan (1918) vacuum-tube trigger circuit. Edge-triggered DFFs became standard with TTL 7474 (1970s). Setup/hold analysis is essential for static timing analysis in modern ASIC flows.

4.7 Metastability and Synchronizers

When setup/hold is violated (e.g. an asynchronous input sampled by a clock), a flip-flop can enter *metastability*—output hovers near $V_{DD}/2$ for unbounded time before resolving to 0 or 1. Probability decays exponentially with extra waiting time. Fix: *two-flop synchronizer* (cascade two DFFs on the receiving clock) gives one extra cycle for resolution; mean time between failures (MTBF) grows by orders of magnitude. Never sample an async signal with a single flop in a reliable system.

4.8 Exercises

- E4.1 **SR forbidden.** For the NOR SR latch, show the $S = R = 1 \rightarrow 0$ transition can go metastable. Why does NAND SR have the same issue with $\bar{S} = \bar{R} = 0$?
- E4.2 **D latch timing.** Sketch C , D , Q for D toggling while $C = 1$ then holding while $C = 0$. Mark where transparency lets a glitch through.
- E4.3 **JK analysis.** Derive $Q^+ = JQ' + \bar{K}Q$ from the JK truth table. Show T is JK with $J = K = T$, and D is JK with $J = D$, $K = \bar{D}$.
- E4.4 **Setup/hold.** A DFF has $t_{su} = 20$ ns, $t_h = 10$ ns, $t_{cq} = 15$ ns. Can a 50 MHz clock drive two DFFs in series without violation? What about 80 MHz?
- E4.5 **Excitation.** Fill the excitation table for D, JK, and T flip-flops for all $Q \rightarrow Q^+$ transitions ($0 \rightarrow 0$, $0 \rightarrow 1$, $1 \rightarrow 0$, $1 \rightarrow 1$).

Chapter 5

Finite State Machines: Mealy, Moore, and Design

“A state machine is memory with a plan.” Combinational logic decides; sequential logic remembers; an FSM decides *based on* what it remembers. Every controller—from vending machine to CPU—is an FSM.

From zero: states from scratch. We define state, transition, and output, then derive Mealy vs. Moore, state diagrams, and systematic synthesis from word problems.

Learning Outcomes

After this chapter you will be able to:

- (L1) define FSM state, transition function, and output function;
- (L2) distinguish Mealy and Moore machines and convert between them;
- (L3) draw state diagrams and tables from word-problem specifications;
- (L4) synthesise synchronous FSMs using D/JK flip-flops;
- (L5) minimise states via partition refinement and handle unused states.

5.1 What Is a Finite State Machine?

An FSM is a 5-tuple $(S, I, O, \delta, \lambda)$: finite state set S , input alphabet I , output alphabet O , next-state function $\delta : S \times I \rightarrow S$, output function λ . On each clock edge, state advances $s^+ = \delta(s, i)$ and output is produced.

Two output styles:

- **Moore:** output depends on state only, $\lambda : S \rightarrow O$ (output changes only on clock edges—glitch-free but

one cycle latency).

- **Mealy:** output depends on state *and* input, $\lambda : S \times I \rightarrow O$ (faster response, but input glitches can propagate).

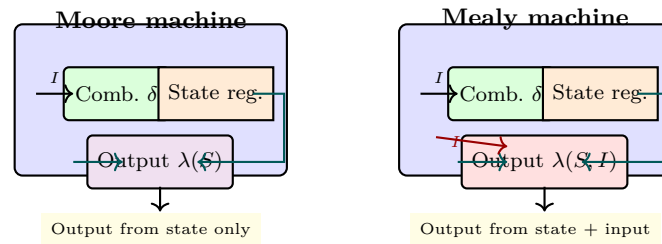


Figure 5.1: Moore vs. Mealy: the only structural difference is whether input feeds the output logic (red arrow). State register is edge-triggered DFFs.

5.2 State Diagrams and Tables

We specify FSMs by *state diagrams* (circles = states, arcs = transitions labelled input/output) or *state tables* (rows = present state, columns = inputs). Example: sequence detector for 101.

Moore: detect 101 (output 1 in S_3)

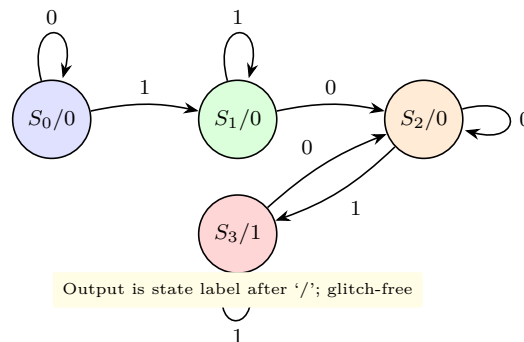


Figure 5.2: Moore FSM that outputs 1 when the last three inputs were 101. State S_i means “last i bits matched prefix of 101”.

The Mealy version needs only 3 states (output on the arc $S_2 \xrightarrow{1/1} S_1$) but its output can glitch if input glitches—tradeoff to choose per application.

5.3 Synchronous Synthesis

Five steps: (1) word problem $\rightarrow$ state diagram, (2) minimise states, (3) encode states in bits, (4) derive next-state and output equations (K-maps), (5) implement with DFFs + combinational logic.

State minimization. Two states are equivalent if no input sequence distinguishes them (same outputs and transitions to equivalent states). Iteratively partition states until stable—the minimal machine is unique up to renaming (like DFA minimization).

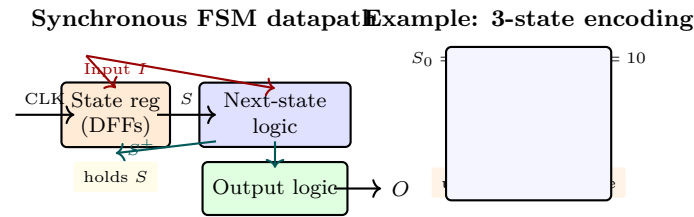


Figure 5.3: Synchronous FSM hardware: state register holds present state; combinational blocks compute next state and output. Unused codes need safe next-state.

Unused states. With k bits you get 2^k codes; if the FSM uses fewer, the remaining codes are don't-cares for minimization but must transition to a safe state (often reset) to avoid lock-up.

5.4 State Encoding and Mealy–Moore Interconversion

Encoding choice affects cost and speed. *Binary* uses $\lceil \log_2 |S| \rceil$ bits (minimal flip-flops, more decoding). *One-hot* uses $|S|$ bits with one 1 (more flip-flops, trivial next-state logic, fast, popular in FPGAs). *Gray* minimizes bit flips between adjacent states, reducing power and glitch risk in output-coded machines.

Mealy $\rightarrow$ Moore conversion: split each state by output value (Moore state = Mealy state + output memory), at most doubling states. Moore $\rightarrow$ Mealy: keep states, move output from state label to incoming arcs—never needs more states.

Example 5.1 (Traffic-light controller). States GREEN, YELLOW, RED with timer inputs. Moore outputs drive lamps directly (glitch-free); Mealy would combine timer expiry and state to anticipate the next lamp—one cycle faster but needs input synchronisation to avoid lamp flicker.

5.5 Chapter Notes

FSM theory originates with Mealy (1955) and Moore (1956), both at Bell Labs. Modern synthesis tools extract FSMs from HDL case statements and optimise encoding (binary, one-hot, Gray) automatically.

5.6 ASM Charts and Algorithmic State Machines

For larger controllers, state diagrams sprawl. An *ASM (Algorithmic State Machine) chart* is a flowchart with three boxes: state box (Moore outputs), decision diamond (input test), conditional output box (Mealy outputs). Each ASM block is one state + its transitions, making translation to HDL (one **case** per state) mechanical.

ASM example: a simple CPU fetch–decode–execute controller has states FETCH, DECODE, EXECUTE with decision diamonds for instruction opcode and conditional outputs for register enables and ALU op codes. Timing is still synchronous—every box executes in one clock, diamond tests are combinational within the state.

Mealy glitch vs. Moore latency. Mealy output can change mid-cycle when input glitches, so any asynchronous input must be synchronized before feeding Mealy output logic. Moore output is glitch-free between

edges but incurs one-cycle latency (input at cycle k affects output at $k + 1$). Many real controllers use Mealy internally for speed and register the output (making it Moore at the pins)—best of both worlds.

Timing of FSM outputs. In a synchronous FSM, next-state is computed combinatorially from present state and inputs, then clocked into DFFs. Output logic runs in parallel. Critical path = $t_{cq} + t_{next} + t_{su}$ (Moore) or $t_{cq} + t_{next,out} + t_{su}$ (Mealy with input-to-output). Pipelining or one-hot encoding shortens t_{next} at the cost of more flops. Static timing analysis checks this path for every state transition.

State reduction example. Two states with identical outputs and transitions to equivalent next states (under current partition) merge without changing I/O behaviour. Repeatedly refine the partition until stable—the result is the minimal machine, unique up to renaming. Lock-up states (no input leads to the valid cycle) must be broken by adding transitions to a safe reset state.

5.7 Exercises

- E5.1 **Mealy vs. Moore.** Convert the Moore 101 detector in Fig. 5.2 to Mealy. How many states? Compare output timing on a timing diagram.
- E5.2 **Vending machine.** Design a Moore FSM for a vending machine: inputs nickel/dime, product costs 15¢, output ‘dispense’. Draw diagram and state table.
- E5.3 **Synthesis.** Synthesise the 3-state FSM in Fig. 5.3 using D flip-flops. Write K-maps for D_1, D_0 and draw the gate-level circuit.
- E5.4 **Minimization.** A FSM has 6 states with given table (provide). Apply partition refinement to find the minimal equivalent machine. Is it unique?
- E5.5 **One-hot vs. binary.** Encode a 4-state FSM in binary (2 bits) and one-hot (4 bits). Compare flip-flop count, next-state logic complexity, and speed.

Chapter 6

Registers and Counters

“Registers hold; counters count; shift registers do both.” Built from flip-flops, these are the first useful sequential subsystems—data storage, sequencing, and serial–parallel conversion in one chapter.

From zero: flip-flops → systems. We assume D/JK flip-flops (Ch. 4) and FSM ideas (Ch. 5). Each circuit is derived from “what next-state do we want?”

Learning Outcomes

After this chapter you will be able to:

- (L1) build parallel registers with load and clear controls;
- (L2) explain shift registers and serial/parallel conversion;
- (L3) design ripple and synchronous counters (binary, decade, up/down);
- (L4) analyse counter timing, modulus, and self-correcting design;
- (L5) distinguish ring and Johnson counters and their decoded outputs.

6.1 Registers: Parallel Storage

A n -bit *register* is n D flip-flops sharing a clock and control signals. Simplest: n DFFs with common CLK—data $D_{n-1} \dots D_0$ latched on the rising edge. Additions: *asynchronous clear* ($C\bar{L}R$ forces 0), *load enable* (LD : when 0, hold; when 1, load), and tri-state outputs for bus connection.

Timing is identical to a single DFF: setup/hold on D relative to CLK, t_{cq} to Q . Registers are the state elements of every FSM and data-path.

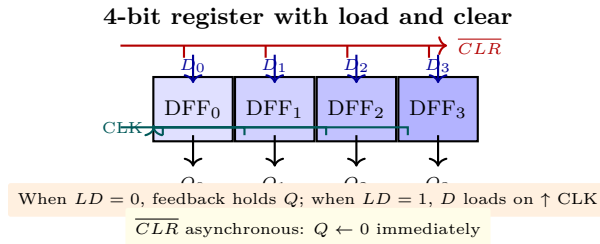


Figure 6.1: Four D flip-flops with common clock, asynchronous clear, and load-hold MUX (conceptually $D_{\text{eff}} = LD \cdot D + \overline{LD} \cdot Q$).

6.2 Shift Registers

Connect Q_i to D_{i+1} and data ripples one position per clock—a *shift register*. Variants: SISO, SIPO, PISO, PIPO (serial/parallel in/out). Add a MUX per stage for *parallel load* vs. shift mode, and a *SI* (serial in) pin on the first stage.

Applications: serial–parallel conversion (UART), delay lines, sequence generators, and multiplication/division by 2 (shift left/right).

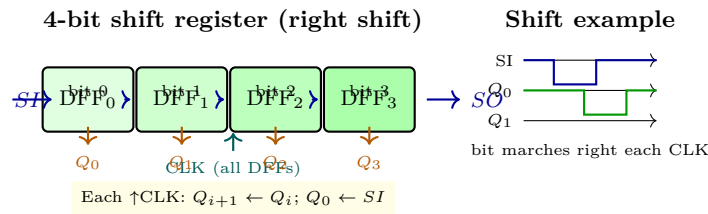


Figure 6.2: Shift register: cascade of DFFs. Serial input enters at the left; parallel outputs tap each stage. Universal shift register adds MUX per stage for load/shift/hold.

6.3 Counters

A *counter* cycles through a sequence of states, usually binary counting. Modulus m = number of distinct states.

Ripple (asynchronous) counter. Chain T flip-flops with Q_i clocking stage $i + 1$. Simple, but delay accumulates and intermediate glitches appear—unsuitable for synchronous decoding. Mod- 2^n from n bits; mod- m ($m \neq 2^n$) needs reset logic when m reached.

Synchronous counter. All flip-flops share CLK and toggle logic is combinational: $T_0 = 1$, $T_1 = Q_0$, $T_2 = Q_1 Q_0$, $\dots$, $T_i = \bigwedge_{j < i} Q_j$. All bits update simultaneously—faster, no glitch, at the cost of AND gates.

Up/down and decade. Add a direction bit to select increment vs. decrement. Decade (0–9) and other non-power-of-two moduli decode terminal count and synchronously reset or use natural BCD behaviour (e.g. 74LS90).

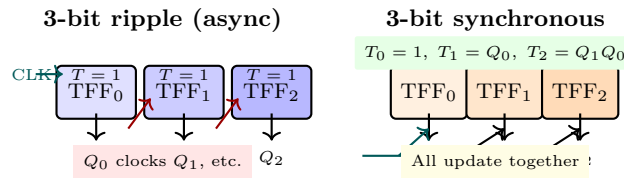


Figure 6.3: Ripple vs. synchronous 3-bit binary counter. Ripple uses the previous Q as clock (red); synchronous shares CLK and gates the T inputs.

Ring and Johnson. *Ring*: shift register with Q_{n-1} fed back to D_0 —one hot circulates (n states, simple decode). *Johnson (twisted ring)*: feed $\bar{Q}_{n-1}$ back— $2n$ states with Gray-like adjacency, decoded by 2-literal ANDs.

6.4 Counter Modulus, Self-Correction, and Timing

Modulus m need not be a power of two. For synchronous mod- m ($m < 2^n$), decode terminal count $m - 1$ and synchronously load 0 (or use natural truncated sequence with don't-cares for $m - 2^n - 1$ to minimize logic). Asynchronous reset for mod- m creates a short glitch state (e.g. $101 \rightarrow 000$ via 101 spike)—use synchronous reset in clocked systems.

Self-correction: every unused state must eventually enter the valid cycle. Verify by tracing next-state from each unused code; if a lock-up cycle exists, add logic to break it (usually force to 0 on next clock). Ring/Johnson counters are inherently self-correcting only if designed with correction gates (e.g. start-up reset).

Example 6.1 (Mod-10 decade). BCD decade 0–9 from 4 bits: states 1010–1111 are don't-cares. K-maps for next-state treat them as $\times$, yielding simpler logic than a full 16-state map. Add synchronous clear when $1001 \rightarrow 0000$ to close the cycle.

6.5 Chapter Notes

Counters and shift registers were the first MSI building blocks (7495, 74193, 74164). Pipelined data-paths still use registers as pipeline boundaries; Johnson counters appear in stepper-motor drivers.

6.6 Applications: From Counters to Timers

Counters plus registers build timers, frequency dividers, and UART baud generators. A down-counter loaded with N and counting to 0 produces a pulse every $N + 1$ clocks (programmable period). A prescaler (power-of-two divider) followed by a programmable counter covers wide timing ranges with modest bit-width. In displays, a BCD counter plus 7-segment decoder drives each digit, multiplexed at ~ 1 kHz so persistence of vision shows a stable number.

Shift-register applications. Beyond storage, shift registers implement *sequence generators* (LFSR with XOR feedback gives maximal $2^n - 1$ pseudo-random sequence), *CRC* computation (polynomial division via

shifted XOR), and *serial protocols* (SPI/I²C shift data on SCLK). The universal shift register's four modes (hold, shift-left, shift-right, parallel load) are selected by two mode bits—one chip, many uses.

6.7 Exercises

- E6.1 **Register timing.** A 4-bit register uses DFFs with $t_{su} = 15$ ns, $t_{cq} = 20$ ns. What is max CLK frequency? What changes with a load MUX adding 8 ns before D ?
- E6.2 **Shift modes.** Draw a 4-bit universal shift register stage with MUX selecting hold/shift-left/shift-right/parallel-load. Give the S_1S_0 table.
- E6.3 **Synchronous design.** Design a mod-6 synchronous binary counter (0–5) using T flip-flops. Derive $T_2T_1T_0$ via K-maps and handle states 6,7.
- E6.4 **Ripple glitch.** Show a 3-bit ripple counter glitch when $011 \rightarrow 100$: list transient binary values and explain why synchronous avoids them.
- E6.5 **Johnson decode.** For a 4-bit Johnson counter, list its 8 states and give minimal decode for each state using 2-input gates.

Chapter 7

Arithmetic Circuits: Fast Adders, Multipliers, and ALUs

“Addition is the mother of all arithmetic.” Fast addition, subtraction, multiplication, and the ALU that unites them—all derived from the full adder and the idea of *anticipating* the carry.

From zero: arithmetic from gates. We assume adders and two’s complement (Ch. 1–3). Each faster circuit is motivated by the delay of the previous one, then derived and costed.

Learning Outcomes

After this chapter you will be able to:

- (L1) compute generate/propagate and derive carry-lookahead equations;
- (L2) design carry-lookahead and carry-select adders and compare delay vs. cost;
- (L3) build array and shift-add multipliers and explain Booth recoding;
- (L4) perform two’s complement subtraction and overflow detection in hardware;
- (L5) describe the ALU organization and status flags (C, Z, N, V).

7.1 Why Ripple Is Too Slow

A 32-bit ripple-carry adder waits for carry to walk 32 stages— ~ 64 gate delays. At 3 GHz that is several clock cycles. Faster adders *compute* carries in parallel instead of waiting for them.

Define for each bit i : generate $g_i = a_i b_i$ (creates a carry), propagate $p_i = a_i \oplus b_i$ (passes an incoming carry), and $c_{i+1} = g_i + p_i c_i$. This recurrence is the key.

7.2 Carry-Lookahead Adder (CLA)

Unroll $c_{i+1} = g_i + p_i c_i$ to get sum-of-products for each carry in terms of g, p and c_0 :

$$\begin{aligned} c_1 &= g_0 + p_0 c_0, \\ c_2 &= g_1 + p_1 g_0 + p_1 p_0 c_0, \\ c_3 &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0, \quad \dots \end{aligned}$$

Each c_i is two gate levels (AND-OR) after g, p are ready— $O(1)$ depth, independent of n , at the cost of fan-in. Practical CLAs are grouped (4-bit blocks with block G, P) and cascaded.

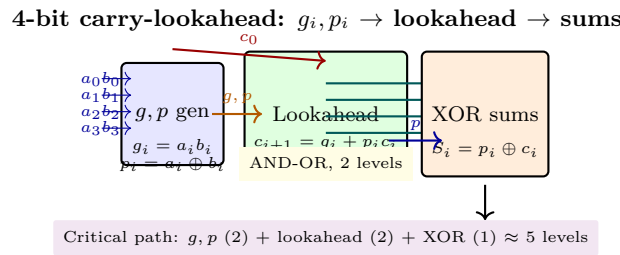


Figure 7.1: Carry-lookahead dataflow. All carries are computed in parallel from g, p, c_0 in two AND-OR levels, then XORed for sums.

Carry-select trades off: compute two speculative results ($c_{\text{mid}} = 0$ and 1) for the upper half and MUX-select when the real carry arrives—less logic than full CLA, faster than ripple.

7.3 Subtraction and Overflow in Hardware

Subtraction $A - B$ is $A + \bar{B} + 1$ (two's complement negate: invert B and set $c_0 = 1$). One adder does both add and subtract with a *SUB* control that XORs B with *SUB* and feeds *SUB* as c_0 . Overflow $V = c_n \oplus c_{n-1}$ (carry into sign $\neq$ carry out) or equivalently, same-sign inputs producing opposite-sign sum.

7.4 Multiplication

Paper-and-pencil binary multiplication is shifts and adds. Two hardware styles:

- **Array multiplier:** $n \times n$ grid of AND gates (partial products) plus adders summing rows— $O(n^2)$ gates, delay $O(n)$.
- **Shift-add (sequential):** one adder + shift register, iterating n cycles—compact, used in microcode.

Booth recoding handles signed two's complement directly by recoding runs of 1s (e.g. 01110 = +16 − 2), reducing partial products and speeding signed multiply.

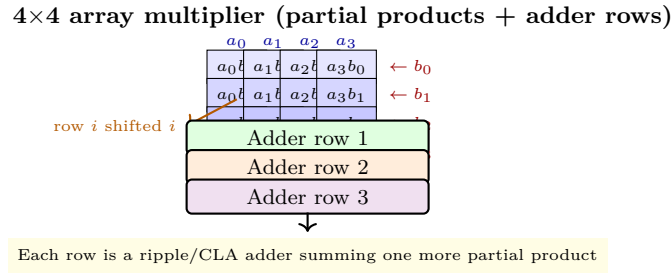


Figure 7.2: Array multiplier: each row of AND gates creates one partial product; successive adder rows accumulate them with increasing shift.

7.5 Arithmetic Logic Unit (ALU)

The ALU is a MUX of operations sharing one adder: inputs A, B , control Op selects arithmetic (+, −, increment) or logic (AND, OR, XOR, NOT) plus shifts. Status flags: C carry out, Z zero (*NOR* of result), N sign (MSB), V signed overflow, used by branch logic.

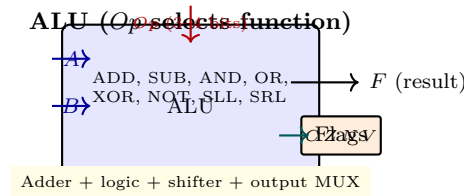


Figure 7.3: ALU organization: one shared adder plus logic and shifter, multiplexed by the operation code; flags summarize the result for branching.

7.6 Carry-Select, Prefix Adders, and Multiplication Tradeoffs

Carry-select partitions n bits into blocks (e.g. $4 + 4 + 4 + 4$ for $n = 16$), precomputes both $c_{in} = 0$ and 1 per block, then MUX-selects. Delay $\approx$ block adder + $(n/\text{blocksize} - 1)$ MUX delays—between ripple and full CLA. *Parallel-prefix* (Kogge–Stone, Brent–Kung) generalises CLA to $O(\log n)$ depth with $O(n \log n)$ or $O(n)$ cost, the standard choice for 32/64-bit ALU adders at GHz frequencies.

Multiplication area–time tradeoff: array is $O(n^2)$ gates but single-cycle; shift-add is $O(n)$ gates but n cycles. For $n = 32$, Wallace/Dadda trees compress partial products with 3:2 counter arrays to $O(\log n)$ stages, used in high-performance multipliers.

Example 7.1 (Flag generation). Result $R = A + B$: C = carry out, $Z = \bigwedge_i \bar{R}_i$ (NOR), $N = R_{n-1}$, $V = C_n \oplus C_{n-1}$. Branch conditions combine them: e.g. signed $A < B$ iff $N \oplus V = 1$ for two’s complement.

7.7 Chapter Notes

CLA was described by Weinberger and Smith (1958); Booth’s algorithm (1951) sped early computer multipliers. Modern ALUs use parallel-prefix adders (Kogge–Stone, Brent–Kung) that generalise CLA to $O(\log n)$ delay.

7.8 Division Preview and Fixed-Point Scaling

Division is the hardest basic operation—iterative subtract-and-shift (restoring or non-restoring) takes n cycles for n bits, with careful handling of two’s complement signs. We omit full design here (computer arithmetic course), but note that multiplication and division by 2^k are just shifts, and fixed-point scaling (e.g. Q15 format: 1 sign + 15 fractional bits) uses this to implement fractional arithmetic with integer hardware, ubiquitous in DSP and embedded control.

Signed overflow vs. carry. Carry C is unsigned overflow; V is signed overflow. For addition, $V = (A_{n-1} \oplus R_{n-1}) \cdot (B_{n-1} \oplus R_{n-1})$ or $C_n \oplus C_{n-1}$. Hardware computes both in parallel from the adder’s internal carries. Branch logic selects which flag to test based on signed vs. unsigned interpretation of the preceding operation (e.g. BCC vs. BVS in 6502/ARM).

Shifter and rotator. A *barrel shifter* rotates or shifts n bits by any amount in $O(\log n)$ MUX stages (e.g. 32-bit shift in 5 stages of 2:1 MUXes controlled by shift-amount bits). Shifts are essential for normalization in floating-point and for alignment in addition. Rotates preserve bits (useful in cryptography); arithmetic right shift replicates the sign bit to preserve two’s complement value.

Multiplication correctness sketch. Array multiplier sums partial products $a_j b_i$ at column $j+i$; each adder row preserves $\sum$ partials = product. By induction on rows, after k rows the partial sum equals $\sum_{i < k} \sum_j a_j b_i 2^{j+i}$, and after n rows the full $2n$ -bit product. Booth’s recoding maintains the same invariant with fewer non-zero partials.

Cost vs. delay summary. Ripple: $O(n)$ gates, $O(n)$ delay. CLA: $O(n^2)$ (naïve) or $O(n)$ with grouping, $O(\log n)$ delay. Carry-select: $\approx 1.5 \times$ ripple gates, $\approx O(\sqrt{n})$ delay. Choose by constraints: area-limited $\rightarrow$ ripple/carry-select, speed-limited $\rightarrow$ prefix CLA.

7.9 Exercises

E7.1 **CLA equations.** For $n = 4$, write $c_1 \dots c_4$ in terms of g_i, p_i, c_0 . Group as 4-bit block with $G = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$, $P = p_3 p_2 p_1 p_0$.

E7.2 **Delay.** Ripple $n = 16$ with $t_{FA} = 2$ vs. CLA with $t_{gp} = 2$, $t_{look} = 2$, $t_{xor} = 1$. Compute worst-case delays and speedup.

E7.3 **Subtractor.** Show how one adder with *SUB* control computes both $A + B$ and $A - B$. Where does overflow V come from? Test $A = 0111$, $B = 0001$.

E7.4 **Booth.** Multiply $6 \times (-3)$ (0101×1101 in 4-bit two’s complement) using Booth recoding. List partial products and final product.

E7.5 **ALU flags.** For $A = 1011$, $B = 0101$, compute $A + B$, $A - B$, $A \& B$ in 4-bit two's complement and give C, Z, N, V for each.

Chapter 8

Memory and Programmable Logic

“Logic computes; memory remembers across power cycles and clock cycles.” From one-bit cells to RAM, ROM, and the programmable fabrics that let you *draw* a circuit into existence—the storage side of digital design.

From zero: memory from latches. We build up from latch/register (Ch. 6) to arrays, then show how any truth table becomes a memory lookup, and how FPGAs generalise this.

Learning Outcomes

After this chapter you will be able to:

- (L1) explain SRAM and DRAM cells and their tradeoffs;
- (L2) describe address decoding, read/write timing, and memory expansion;
- (L3) distinguish ROM, PROM, EPROM, EEPROM, and Flash;
- (L4) use PLA, PAL, and ROM as two-level logic implementations;
- (L5) describe FPGA organization (LUTs, interconnect, configuration).

8.1 Memory Organization

A $2^n \times m$ memory holds 2^n words of m bits. Inputs: n address lines select one word, m data lines carry the word, plus $\overline{CS}$ (chip select), $R/\overline{W}$ (read vs. write), and $\overline{OE}$ (output enable). Internally: a matrix of cells, row/column decoders, and sense amplifiers.

Capacity examples: $1\text{K} \times 8 = 8192 \text{ bits} = 1 \text{ KB}$. Address space grows exponentially with n ; widening m replicates arrays in parallel.

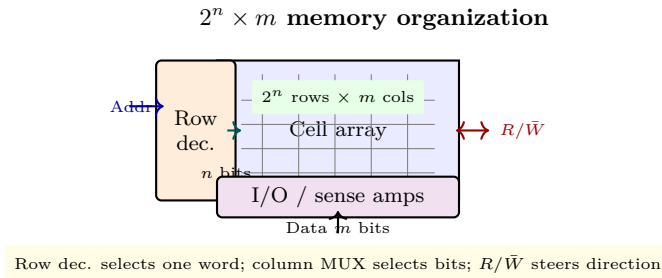


Figure 8.1: Memory as a matrix: address decoded to one row, data flows through sense amplifiers. Controls $R/\bar{W}$, $\overline{CS}$, $\overline{OE}$ gate the transfer.

8.2 SRAM vs. DRAM Cell

SRAM cell: 6-transistor bistable (cross-coupled inverters + access transistors)—fast, no refresh, larger area.

DRAM cell: 1 transistor + 1 capacitor—tiny, dense, but charge leaks and must be refreshed every few ms, and reads are destructive (charge sharing) so a write-back is required. SRAM for caches, DRAM for main memory.

ROM variants: mask ROM (factory), PROM (one-time fuse), EPROM (UV-erase), EEPROM/Flash (electrically erase in blocks)—non-volatile.

8.3 Programmable Logic: PLA, PAL, ROM

Any n -input, m -output combinational function is a $2^n \times m$ lookup table: address = input code, data = output values. Using a ROM this way trades silicon for universality but wastes capacity when don't-cares abound.

PLA (Programmable Logic Array): programmable AND plane (product terms) followed by programmable OR plane—minimal two-level realization. *PAL* (Programmable Array Logic): programmable AND, fixed OR—cheaper, faster, slightly less flexible. Both are field-programmable (fuses/antifuses/Flash).

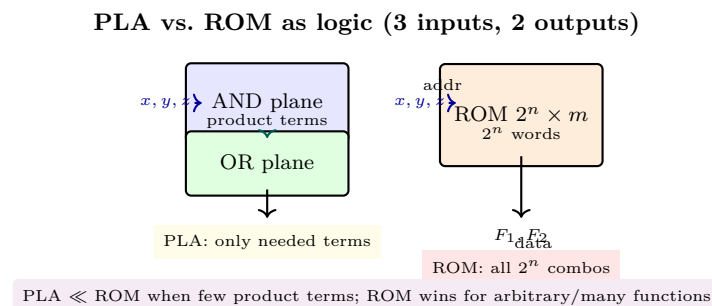


Figure 8.2: Two ways to make any combinational function: PLA creates only the product terms you need; ROM stores the full truth table.

8.4 FPGAs: Logic You Configure

A Field-Programmable Gate Array tiles *Configurable Logic Blocks* (CLBs) containing Look-Up Tables (LUTs, typically 4–6 input SRAM memories that *are* tiny ROMs), flip-flops, and MUXes, stitched by a programmable interconnect fabric. Configuration is a bitstream loaded from Flash/SRAM that sets every LUT truth table

and every routing switch.

Design flow: HDL $\rightarrow$ synthesis $\rightarrow$ mapping to LUTs $\rightarrow$ place & route $\rightarrow$ bitstream. Reprogrammable in milliseconds—prototyping, acceleration, and low-volume production.

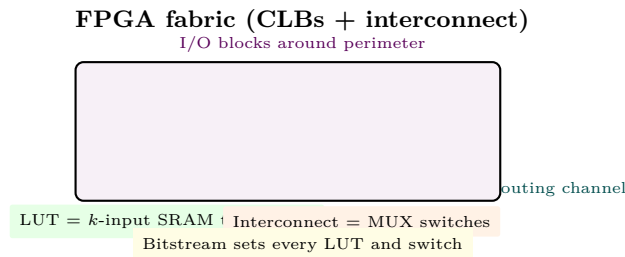


Figure 8.3: FPGA: array of CLBs (each a LUT plus optional flip-flop) connected by a programmable routing fabric. The bitstream configures LUT contents and interconnect.

Memory hierarchy note: registers (fastest, fewest), SRAM caches, DRAM main memory, Flash/SSD storage—each level trades speed for capacity and cost. Temporal and spatial locality make the hierarchy effective.

8.5 Read/Write Timing and Memory Expansion

Read cycle: place address, assert $\overline{CS}$ and $\overline{OE}$, wait t_{AA} (address access time), sample data. Write cycle: place address and data, pulse $\overline{WE}$ or $R/\overline{W}$, meet setup/hold t_{AS}, t_{DS}, t_{DH} . SRAM is asynchronous or synchronous (clocked, pipelined for cache). DRAM needs $\overline{RAS}/\overline{CAS}$ multiplexing and periodic refresh (burst or distributed, ~ 64 ms retention).

Expansion: widening (m) parallels chips sharing address; deepening (2^n) decodes upper address bits to chip-selects. Example: $4K \times 8$ from $1K \times 8$ needs 4 chips, 2 address bits into a 2-to-4 decoder whose outputs are $\overline{CS}_i$, data buses tied together (tri-state or wired-OR with $\overline{OE}$ gating).

Example 8.1 (ROM as lookup). A 32×8 ROM stores ASCII codes: address = 5-bit input code, data = 8-bit ASCII. Changing the function is a ROM reprogram, not a rewiring—the original “firmware” idea before Flash and FPGAs made reconfiguration routine.

8.6 Chapter Notes

Wilkes (EDSAC, 1949) introduced stored-program memory; Dennard’s 1T DRAM (1968) enabled cheap main memory. PLAs/PALs dominated glue logic in the 1980s; Xilinx XC2064 (1985) launched the FPGA era (now AMD/Xilinx and Intel/Altera).

8.7 Memory Hierarchy and Locality

Registers ($t \sim 0.3$ ns) $\rightarrow$ SRAM cache (1–3 ns) $\rightarrow$ DRAM (50–100 ns) $\rightarrow$ Flash/SSD (50 μ s–1 ms) $\rightarrow$ disk. Each level is larger, cheaper per bit, and slower. *Temporal locality* (reuse) and *spatial locality* (nearby addresses) make caches effective: a small fast cache captures most accesses. Write policies (write-through vs.

write-back) and replacement (LRU) manage coherence between levels—full treatment in computer architecture (SC1007/SC2002).

Error detection. Memories add parity or ECC: single-bit parity detects one error; Hamming (72, 64) corrects one bit and detects two (SECDED) with 8 check bits per 64-bit word—standard in servers. Flash adds wear leveling and bad-block management since cells degrade after $\sim 10^5$ program/erase cycles; the controller remaps writes to spread wear, turning the memory into a small computer itself.

Non-volatile evolution. Mask ROM gave way to PROM (fusible links), EPROM (floating-gate, UV erase), EEPROM (byte-erase via Fowler–Nordheim tunneling), and Flash (block-erase, NOR for XIP vs. NAND for density). Emerging NVM—MRAM, ReRAM, PCM—promise DRAM-like speed with non-volatility, potentially collapsing the memory hierarchy in future architectures. Resistance and endurance remain active research frontiers.

Programmable logic in modern flows. HDL synthesis maps `case` and `if-else` to MUXes and LUTs automatically; the designer rarely instantiates PLA/PAL explicitly. Yet understanding them explains why a 4-LUT FPGA needs $\lceil n/4 \rceil$ LUTs per n -input function and why wide AND-OR structures benefit from dedicated carry/AND chains in modern FPGAs (e.g. Xilinx CARRY4).

8.8 Exercises

- E8.1 **SRAM vs. DRAM.** Compare 6T SRAM and 1T1C DRAM on transistor count, speed, refresh, and use case. Why is DRAM destructive on read?
- E8.2 **Expansion.** Build $2K \times 8$ from $1K \times 8$ chips. How many chips? Draw address and data wiring, including $\overline{CS}$ decoding.
- E8.3 **ROM logic.** Implement $F = \sum m(1, 2, 5, 6)$ and $G = \sum m(0, 3, 7)$ (3 inputs) using an 8×2 ROM. List ROM contents. Repeat with a PLA using minimal product terms.
- E8.4 **LUT mapping.** A 4-LUT can realise any 4-input function. How many 4-LUTs are needed for a 6-input AND? For a 4-bit adder’s sum bits? Sketch the mapping.
- E8.5 **Flash vs. EEPROM.** Distinguish NOR and NAND Flash on erase granularity, random access, and typical application (code storage vs. data storage).